

# Next Generation Reservoir Computing

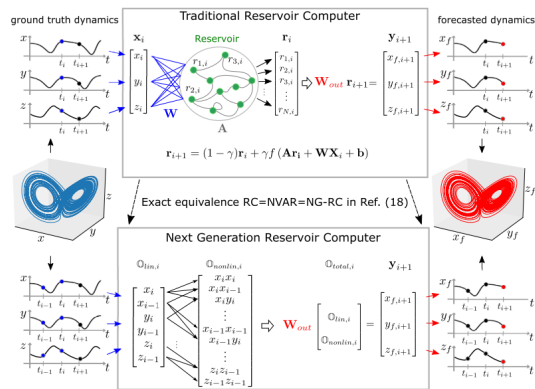
From recurrent neural reservoirs to nonlinear vector autoregression

Based on Gauthier, Bollt, Griffith & Barbosa (2021)

Nature Communications 12, 5564 (2021)

<https://doi.org/10.1038/s41467-021-25801-2>

# Traditional Reservoir Computing: the basic picture



- ▶ The input time series  $\mathbf{X}_i$  is pushed into a large recurrent network: the **reservoir**.
- ▶ The reservoir state  $\mathbf{r}_i$  mixes current input with memory of previous inputs.
- ▶ We do **not** train the recurrent network. The random matrices stay fixed.
- ▶ Only the readout matrix  $\mathbf{W}_{out}$  is fitted.

Source: Figure 1 of the paper, p. 2.

# What is the reservoir actually doing?

The reservoir evolves according to

$$\mathbf{r}_{i+1} = (1 - \gamma)\mathbf{r}_i + \gamma f(A\mathbf{r}_i + W\mathbf{X}_i + b) \quad (1)$$

- ▶  $\mathbf{r}_i \in \mathbb{R}^N$ : the states of the  $N$  reservoir nodes.
- ▶  $A$ : reservoir-to-reservoir connectivity matrix.
- ▶  $W$ : input-to-reservoir weights.
- ▶  $f(\cdot)$ : nonlinear node activation, commonly tanh.
- ▶  $\gamma$ : decay/leak parameter.
- ▶  $b$ : bias vector.

## Why the reservoir works

Every new state depends on both the current input and the previous reservoir state. In other words, the network carries a fading memory of the input history.

Notation follows Eq. (1) in Gauthier et al.;  $i$  indexes discrete time steps.

## The part we actually train: the readout

The output is a linear combination of features extracted from the reservoir:

$$Y_{i+1} = W_{\text{out}} \mathcal{O}_{\text{total},i+1} \quad (2)$$

For supervised training, the paper uses Tikhonov (ridge) regularization:

$$W_{\text{out}} = Y_d \mathcal{O}_{\text{total}}^T (\mathcal{O}_{\text{total}} \mathcal{O}_{\text{total}}^T + \alpha I)^{-1} \quad (3)$$

### Why regularize?

The parameter  $\alpha$  discourages excessively large readout weights. That reduces overfitting and also makes the fit more tolerant of noise.

### NOTE

Training is still just linear algebra. There is no backpropagation through the entire recurrent network.

Source: Eqs. (2)–(3), p. 3.

# From this construction to NVAR

## The central mathematical result

A linear reservoir with a polynomial nonlinear readout is mathematically equivalent to a **Nonlinear Vector Autoregression (NVAR)**.

### Traditional RC

$\mathbf{r}_i \xrightarrow{\text{recurrent dynamics}} \text{hidden state} \xrightarrow{\text{readout}} Y_{i+1}$

- ▶ No random reservoir matrix is needed.
- ▶ There are fewer metaparameters to search over.
- ▶ The features are explicit, so we can inspect what the model is using.

### NG-RC / NVAR

time-delayed data  $\xrightarrow{\text{nonlinear features}} \mathcal{O}_{\text{total},i} \xrightarrow{W_{\text{out}}}$

Source: Equivalence and existence-proof discussion, pp. 2 and 4.

# The NG-RC feature vector

The model combines a constant term, delayed linear observations, and nonlinear polynomial terms:

$$\mathcal{O}_{\text{total}} = c \oplus \mathcal{O}_{\text{lin}} \oplus \mathcal{O}_{\text{nonlin}} \quad (4)$$

For the Lorenz63 experiment, the paper uses a quadratic construction:

$$\mathcal{O}_{\text{total}} = c \oplus \mathcal{O}_{\text{lin}} \oplus \mathcal{O}_{\text{nonlin}}^{(2)}, \quad (5)$$

so that

$$\dim(\mathcal{O}_{\text{total}}) = 1 + dk + \frac{dk(dk + 1)}{2}.$$

For  $d = 3$  and  $k = 2$ :

$$1 + 6 + \frac{6 \times 7}{2} = 28.$$

## Compare this with a reservoir

The NG-RC uses a feature vector of only 28 components in this experiment, rather than a recurrent network with hundreds or thousands of nodes.

# Benchmark I: the Lorenz63 system

The first test is the classic Lorenz system:

$$\begin{aligned}\dot{x} &= 10(y - x), \\ \dot{y} &= x(28 - z) - y, \\ \dot{z} &= xy - \frac{8}{3}z.\end{aligned}\tag{6}$$

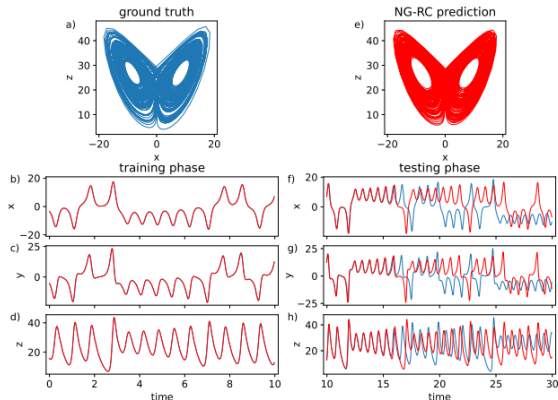
- ▶ Three coupled nonlinear differential equations.
- ▶ Deterministic, but chaotic.
- ▶ Nearby initial conditions separate rapidly.
- ▶ The trajectory settles onto the familiar two-wing strange attractor.

## Why is this a useful test?

A successful model must do more than interpolate the training series. When run autonomously, it should reproduce both short-term behavior and the long-term geometry of the attractor.

Source: Eq. (7) and Lorenz63 discussion, p. 4.

# Lorenz63: from 400 samples to an autonomous forecast



- ▶ 400 data points per variable.
- ▶  $dt = 0.025$ ,  $k = 2$ ,  $s = 1$ .
- ▶  $\alpha = 2.5 \times 10^{-6}$ .
- ▶ 28 total features, giving  $W_{out} \in \mathbb{R}^{3 \times 28}$ .

## What do we see?

The predicted attractor closely follows the two-wing structure of the true Lorenz attractor. The autonomous forecast remains accurate for about 5 Lyapunov times.

Training NRMSE:  $1.06 \pm 0.01 \times 10^{-4}$  over one Lyapunov time.

## Benchmark II: the double-scroll electronic circuit

The second system is more awkward mathematically because its vector field is non-polynomial:

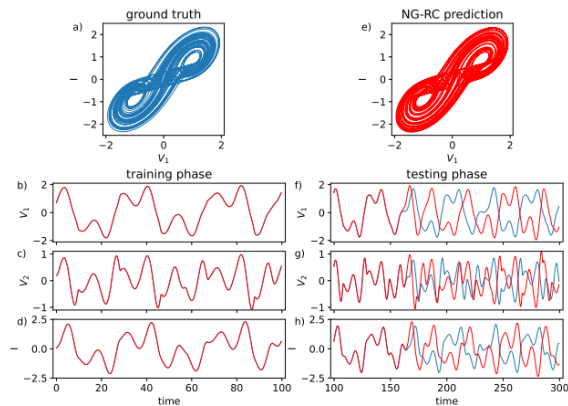
$$\begin{aligned}\dot{V}_1 &= \frac{V_1}{R_1} - \frac{\Delta V}{R_2} - 2I_r \sinh(\beta \Delta V), \\ \dot{V}_2 &= \frac{\Delta V}{R_2} + 2I_r \sinh(\beta \Delta V) - I, \\ \dot{I} &= V_2 - R_4 I,\end{aligned}\quad \Delta V = V_1 - V_2. \quad (7)$$

- ▶ Here the nonlinearity enters through  $\sinh(\beta \Delta V)$ .
- ▶ Yet the NG-RC still uses polynomial features.
- ▶ The paper deliberately chooses this example to test whether a low-order polynomial feature map can still capture a non-polynomial system.

### Feature choice

Because the attractor has odd symmetry and zero mean for the chosen parameters, the model uses linear + cubic terms rather than including a constant and quadratic terms.

# Double-scroll: the polynomial model still works



- ▶ Same basic training-set size: 400 points per variable.
- ▶  $dt = 0.25$ ,  $k = 2$ ,  $s = 1$ .
- ▶ Cubic nonlinear features.
- ▶ 62 total features.

## Result

The reconstructed attractor and the time-series forecast are qualitatively close to the ground truth.

Forecasting NRMSE:  $4.5 \pm 1.0 \times 10^{-3}$ .

The important point is not that a cubic polynomial reproduces  $\sinh$  exactly; it does not. The model is learning a finite-step flow from data rather than symbolically recovering the original differential equations.

Source: Figure 3 and discussion, pp. 4–5.

## Benchmark III: can we infer a variable we cannot measure?

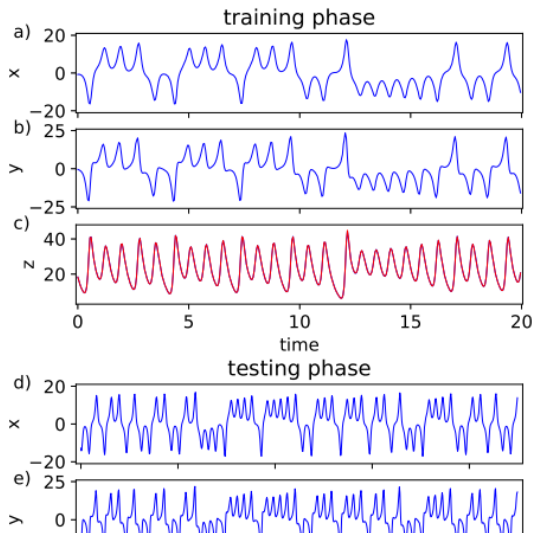
**Training:** the model sees  $x, y, z$ .

**Testing:** only  $x, y$  are supplied; the goal is to infer  $z$ .

### Why delay coordinates help

Takens' embedding theorem motivates reconstructing the hidden dynamics from delayed observations. In practice, several past measurements can contain enough information to reconstruct the state-space geometry.

- ▶  $k = 4, s = 5, dt = 0.05$ .
- ▶ Only 400 training points are used.
- ▶ 45 total features;  $W_{out}$  is  $1 \times 45$ .



# What do the experiments establish?

## 1. Short-term forecasting

The NG-RC can learn accurate one-step dynamics and remain useful over multiple Lyapunov times on chaotic systems.

## 2. Long-term climate / attractor reconstruction

Even when exact trajectories eventually diverge—as expected in chaotic systems—the generated trajectory can still occupy nearly the same attractor geometry.

## 3. Inference from partial observations

The method can use delay-coordinate information to infer a variable that is not provided during testing.

Source: Benchmark summary, pp. 3–6.

# Why is NG-RC computationally attractive?

| Aspect                   | Traditional RC                           | NG-RC in this paper                                                                                                                       |
|--------------------------|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Internal representation  | Random recurrent reservoir               | Explicit delay + polynomial features                                                                                                      |
| Design choices           | Random matrices + several metaparameters | Mainly delay, spacing, polynomial order, $\alpha$                                                                                         |
| Warm-up                  | Can be very long                         | Determined by the delay embedding                                                                                                         |
| Training                 | Regularized linear readout               | Regularized linear regression on explicit features                                                                                        |
| Lorenz63 simulation cost | Large recurrent-state update             | About $33\text{--}162\times$ less costly than a typical efficient RC; $> 10^6\times$ less than a high-accuracy RC in the paper's estimate |

# Where the paper is careful—and where more work is needed

## What was actually tested

- ▶ Low-dimensional dynamical systems.
- ▶ Main-text experiments use noise-free numerical simulations.
- ▶ Polynomial features of relatively low order.

## What the authors point toward

- ▶ Feature selection and sparsification.
- ▶ Kernel methods for very large feature spaces.
- ▶ Noisy-data applications with stronger regularization.
- ▶ High-dimensional spatiotemporal systems.
- ▶ Digital twins, data assimilation, nonlinear control, and fast simulation.

Source: Discussion, pp. 6–7.

## So, what is the main idea?

### **A reservoir can be implicit.**

Instead of explicitly building a large recurrent neural network, the NG-RC builds a compact feature vector directly from the recent history of the observed system:

recent observations  $\rightarrow$  delay embedding  $\rightarrow$  nonlinear features  $\rightarrow$  ridge regression  $\rightarrow$  one-step prediction (8)

- ▶ The representation is explicit rather than hidden inside a random network.
- ▶ The learned map can be iterated autonomously.
- ▶ For the benchmark systems in the paper, very small training sets are enough.

### The conceptual shift

We do not necessarily need to simulate the complicated internal reservoir if the same useful computation can be written directly as a nonlinear autoregressive model.

## Primary source used for this presentation

The attached paper by Gauthier et al. is the main source for the equations, benchmark settings, figures, and computational comparisons presented here.